Few-View Computed Tomography — Run Instructions

Step-by-step procedure for reproducing the supplied matrix-free CT benchmark

1. Before starting

The benchmark is intended to be run on a machine or computing service with enough memory and CPU resources for a 1024 by 1024 matrix-free CT calculation. The dissertation run was a large-scale computing experiment. The repository does not require the generated CSV or PNG outputs to be stored permanently because they are produced by the programs.

The reader should first make sure that Python is available and that the required scientific Python packages are installed. The exact installation method depends on the machine being used. On a University computing service, the current University documentation should be followed for logging in and for creating or activating a suitable Python/Anaconda environment.

2. Files required

File	Needed for
CT.py	Numerical benchmark
plot_CT.py	Plot generation after the benchmark

Keep both Python files in the same working directory when following the simple commands below. The plotting program expects the CSV files produced by CT.py to be inside the directory named CT_real_world_output.

3. Running on a normal Python/Anaconda installation

The following procedure is suitable for a local installation with sufficient resources.

Step 1 — Open a terminal or Anaconda Prompt. On macOS or Linux, Terminal can be used. On Windows, Anaconda Prompt or another terminal can be used.

Step 2 — Move into the directory containing CT.py. For example, if the files are in a folder called CT, use a command of the following form:

```
cd /path/to/CT
```

Step 3 — Confirm that Python can be called:

```
python --version
```

If the system uses the command python3 instead, use python3 in the commands below. The important point is to use the Python interpreter belonging to the environment in which NumPy, SciPy, pandas and Matplotlib are installed.

Step 4 — Run the benchmark with the supplied default settings:

```
python CT.py
```

The program prints the benchmark configuration and then reports progress for each regularisation value, repeat and solver. The calculation is substantial. The terminal should not be closed while it is running.

4. What happens during CT.py

The program first creates the 1024 by 1024 reference phantom and the matrix-free parallel-beam CT operator. It calculates the clean projection data and then adds controlled noise. For each value of lambda, it constructs the PCG preconditioner and runs CG, PCG, LSQR and LSMR for the two repeats.

At the end, a directory called CT_real_world_output is created if it does not already exist. Three CSV files and one metadata text file are written there:

Output	Contents
CT_solver_results.csv	Individual solver results for each lambda and repeat.
CT_solver_summary.csv	Mean and standard-deviation results grouped by lambda and solver.
CT_solver_runtime_comparison.csv	Runtime and iteration comparisons involving LSQR.
CT_experiment_metadata.txt	Benchmark configuration and method description.

5. Checking that the numerical run finished

A completed run should end with a COMPLETE message and list the generated files. The output directory should contain the four files listed above. The exact runtime printed by the program will depend on the hardware and system load.

If the process stops with a memory error, the problem is not normally fixed by changing a solver name. The CT calculation is deliberately large. The requirements document in this folder should be read before attempting to reduce or increase resource requests.

6. Running on the CSF or another Slurm system

For a large-scale reproduction, the benchmark can be submitted as a Slurm job rather than kept in an interactive terminal session. The procedure for logging into the CSF must be learned from the University's current CSF documentation; this document does not attempt to reproduce the University's login procedure because those details can change.

Once logged in, the Python files should be copied into a suitable working directory on the CSF. The Python environment must then be selected or activated according to the current CSF software instructions.

A Slurm submission script can be used to request the required resources and run CT.py. The exact partition, account, module names and environment commands are University-dependent and should be checked against the current CSF documentation. The important benchmark-specific point is that the dissertation experiment was run as a large batch calculation rather than as a normal laptop calculation.

The practical memory requirement is especially important. For the direct-solver cases elsewhere in the dissertation, allocations below 128 GB were insufficient. For this CT experiment, the supplied implementation is matrix-free and does not explicitly construct the CT matrix, but the 1024 by 1024 calculation still requires substantial memory. A researcher should check the University's current memory allocation guidance and request an allocation appropriate to the actual implementation rather than assuming that a small default job is sufficient.

The number of CPUs must also be treated as user- and system-dependent. The researcher should check the current CSF allocation limits and select an appropriate CPU request. Increasing CPU count does not automatically reproduce a previous runtime, because the available hardware and parallel execution conditions may differ.

7. Example Slurm structure

The following is a template showing the structure of a batch submission. It is deliberately not tied to a historical job number, account, partition or University-specific module name.

```
#!/bin/bash
#SBATCH --job-name=ct_benchmark
#SBATCH --time=24:00:00
#SBATCH --mem=128G
#SBATCH --cpus-per-task=<check-current-allocation>

# Load or activate the Python environment using the
# current CSF instructions.

python CT.py
```

The 24-hour value reflects the allocation used for the dissertation-style batch run. The memory and CPU directives must be checked against the current University's rules before submission. In particular, the CPU count is not a fixed property of the benchmark.

If an array or multiple independent jobs are used, the same benchmark parameters must be retained for every intended case. The supplied CT.py itself performs the lambda sweep and repeats, so a separate Slurm array is not necessary merely to reproduce the supplied script.

8. Running the plotting program

The plotting program should be run only after CT.py has successfully produced the three CSV files. Change into the directory containing the CSV output directory and plot_CT.py, then run:

```
python plot_CT.py
```

The script reads CT_solver_summary.csv, CT_solver_runtime_comparison.csv and CT_solver_results.csv from the current directory. If those files are inside CT_real_world_output, move into that directory before running the plotting program or copy the plotting script there. The simplest arrangement is to place plot_CT.py in the CT_real_world_output directory and run it from there.

Step-by-step, the plotting stage is therefore: (1) locate CT_real_world_output; (2) confirm the three CSV files are present; (3) place plot_CT.py in that directory or otherwise run it with the correct working directory; (4) execute python plot_CT.py; and (5) open the newly created CT_plots directory.

9. Plot outputs

File	Figure represented
01_runtime_vs_lambda.png	Mean runtime against lambda.
02_reconstruction_error_vs_lambda.png	Mean relative reconstruction error against lambda.
03_residual_vs_lambda.png	Mean relative data-fit residual against lambda.
04_normal_residual_vs_lambda.png	Mean relative normal-equation residual against lambda.
05_best_reconstruction_error_bar.png	Best observed reconstruction error for each solver.
06_fastest_runtime_bar.png	Fastest observed mean runtime for each solver.
07_lsqr_runtime_ratio.png	LSQR runtime comparison at the first lambda in the runtime table.
08_iterations_vs_error.png	Mean iteration count against reconstruction error.
09_runtime_accuracy_tradeoff.png	Runtime against reconstruction error.
10_runtime_grouped_bar.png	Grouped runtime comparison across lambda values.
plot_summary.txt	Short text summary of the plotting inputs and selected rows.

10. Retrieving results from CSF

After a Slurm job has finished, the CSV files and CT_plots directory remain in the job's working directory unless the job script specifies another output location. The University's current recommended file-transfer method should be used to copy the files back to a local computer.

The files needed for checking the result are CT_solver_results.csv, CT_solver_summary.csv, CT_solver_runtime_comparison.csv and the contents of CT_plots. The metadata file CT_experiment_metadata.txt is also useful because it records the principal benchmark settings.

11. Common problems

Python cannot find a package: activate the correct Python/Anaconda environment or install the missing package in accordance with the environment's package-management policy.

FileNotFoundError when plotting: check that the plotting program is being run from the directory containing the three CSV files.

Memory failure: check the requested and available memory. Do not interpret a reduced-size test as the dissertation benchmark.

Slurm job does not start: check the current CSF account, partition, time, memory and CPU rules. These are University infrastructure settings and can change.

Runtime differs from the dissertation: this is expected when hardware, CPU allocation, Python/SciPy versions, system load or other execution conditions differ.

12. Final reproducibility check

Before describing a run as a reproduction, verify that the 1024 by 1024 image, 64 views, 65,536 measurements, five lambda values, noise level 0.002, two repeats, tolerance 1e-6, maximum of 120 iterations, four solvers and matrix-free operator are unchanged. Record the software environment and computational resources used for the new run.